Chapitre 9

Algorithmes pour l'intelligence artificielle.

Définition 1: Apprentissage machine.

En apprentissage machine, on montre à la machine un grand jeu de données d'entraînement, et elle renvoie une fonction pour notre problème. On vérifie la qualité de cette fonction en la comparant à un nouveau jeu de données.

Définition 2: Apprentissage supervisé.

Dans l'apprentissage supervisé, on utilise un jeu de données étiqueté, chaque élément est composé d'une intance, et d'une étiquette indiquant le résultat attendu.
Un exemple : une image avec un texte la décrivant, ou un temps t et une température.

Définition 3

On distingue deux types de problèmes :

- Les problèmes de régression où l'ensemble des étiquettes est continu.
- Les problèmes de classification où l'ensemble des étiquettes est discret.

On se restreint aux problèmes de classification.

Définition 4: Algorithme des k plus proches voisins.

Étant donné une instance, on la compare à celles du jeu d'entraînement pour déterminer lesquelles lui ressembles le plus. On garde les k plus proches. On renvoie alors l'étiquette la plus représentée parmi ceux-ci. La performance de l'algorithme dépend de la distance choisie pour calculer les plus proches voisins, et de k.
Lorsque k est petit, on ne considère que les voisins très proches, on est très sensible aux données d'entraînement. Il n'y a pas eu de généralisaiton, on parle de sur-apprentissage.
Lorsque k est trop grand, on considère des voisins qui ne sont plus représentatifs, on parle de sous-apprentissage.

Exemple 5

Étant donné un nouvel élément, comment calculer ses k plus proches voisins ?

Solution :

Première solution.
Calculer la distance de notre nouvel élément à chacune des donénes, trier les données selon la distance, renvoyer les k premiers, donc $O(n \log n)$.
Seconde solution.
On stocke les k meilleurs candidats, et pour chaque nouvel élément, on le compare au moins bon pour décider de le garder ou non.
Avec un tableau on a une complexité en $O(nk)$. Avec une file de priorité, en $O(n \log k)$.
En construisant un tas sur toutes les données : $O(n + k \log n)$. **Troisième solution.** En $O(n + k)$...

Définition 6: En $O(n+k)$.

On cherche à écrire un algorithme qui renvoie le $k^{\text{ème}}$ plus petit élément d'un tableau. Une fois qu'on a cet élément, il suffit de parcourir toutes les données en ne conservant que les plus proches.

Algorithme 1 : Algorithme A.

Entrées : T un tableau, k le rang de l'élément recherché.
Couper le tableau en morceaux de taille 5.
pour chaque morceau **faire**
| Calculer sa médiane.
fin
Construire T_1 rassemblant ces médianes.
Calculer x la médiane de T_1 avec $A(T_1, \frac{|T_1|}{2})$.
Partitionner T en deux tableaux T' et T'', où T'' contient les éléments plus grands que x.
Si $|T'| = k$, on renvoie x, si $|T'| > k$, on renvoie $A(T', k)$, sinon $A(T'', k - |T'| - 1)$.
Si $|T| < 5$, on le trie et on renvoie le k-ème élément de T.

On a transformé le problème de trouver les k plus proches voisins en trouver les k plus petits éléments d'un tableau.

Justification :

Terminaison.
Chaque appel récursif est réalisé sur un tableau strictement plus petit, les opérations qui ne sont pas des appels récursifs terminent.
Correction.
Par récurrence sur $|T|$. Pour $|T| < 5$ c'est bon.
Supposons la correction sur tous les tableaux de taille inférieure à n. Soit T un tableau de taille $n+1$.
Chacun des appels récursifs est effectué sur un tableau strictement plus petit donc l'appel récursif est correct.
Les tableaux T_1 et T_2 contiennent tous les éléments de T sauf x.
— Si $|T_1| \geq k$, alors le k-ième élément de T est aussi celui de T_1.
Dans ce cas-ci l'algorithme renvoie bien le bon élément.
Complexité.
— Calcul de la médiane des sous-tableaux : $O(n)$.
— Appel récursif sur T' : $C(\frac{n}{5})$ où C est la complexité de l'algorithme.
— Partitionner T et T' : $O(n)$.
— Appel récursif sur T' ou T'' : $C(\max(|T'|, |T''|)) \leq C(\frac{7n}{10})$.
Alors $C(n) \leq C(\frac{n}{5}) + C(\frac{7n}{10}) + n$.
Par récurrence, on montre $C(n) \leq \alpha n$. C'est vrai pour $n = 1$.
Alors $C(n) \leq n(\frac{9}{10}\alpha + 1)$. On pose $\alpha = 10 + C(1)$, alors :
On a $C(1) \leq \alpha$, et pour $n \in \mathbb{N}$ tel que $\forall k < n,\ C(k) \leq \alpha n$:
$C(n) \leq \frac{9\alpha}{10}n + n \leq 9n + n + \frac{9C(1)}{10}n \leq n(10 + \frac{9C(1)}{10}) \leq n(10 + C(1)) \leq n\alpha$.

Méthode des clusters (kd-arbres) pour obtenir les 2-plus proches voisins (MP2I DS2).
Avec cette méthode, l'entraînement n'est pas superficiel puisqu'il consiste à construire l'arbre kd.
Si le nombre de dimensions est grand, de l'ordre du nombre d'éléments, alors la distance à l'hyperplan est souvent petite. Donc l'algorithme va explorer beaucoup plus de noeuds. On parle de phénomène de la malédiction des grandes dimensions.

Définition 7

Pour mesurer la qualité d'un algorithme d'appretissage supervisé, on utilise un jeu de tests indépendant du jeu d'entraînement. On peut évaluer le taux de réussite de l'algorithme sur une étiquette donnée.

Définition 8: Arbres de décision.

Un arbre de décision est un arbre dont les feuilles sont des étiquettes et les noeuds internes des questions relatives aux données.
Exemple. Clés de détermination pour les arbres (plantes).
La création de l'arbre constitue donc l'apprentissage. On veut poser des questions dont les réponses séparent bien les différentes étiquettes. Pour mesurer à quel point un ensemble est déséquilibré, on utilise l'entropie de Shannon. Celle-ci mesure la quantité de connaissances que l'on a sur un ensemble :

$$H(S) = -\sum_{x \in X} f(x) \log_2(f(x))$$

où X est l'ensemble des étiquettes, et $f(x)$ la proportion d'étiquettes x dans S.
Dans le cas $|X| = 2$, H est minimale en $f(x) \in \{0, 1\}$ et maximale en $f(x) = \frac{1}{2}$.
De plus, H est croissante puis décroissante, et symétrique en $1/2$.
On veut minimiser $\sum_{S_i} \frac{|S_i|}{|S|} H(S_i)$ où les S_i sont les ensembles partitionnés par une question.

Exemple 9: ID3

Entrées : Ensembles S de données et Q de questions
si $Q = \varnothing$ **alors**
| **retourner** la feuille avec l'étiquette la plus représentée dans S.
fin
si $S = \varnothing$ **alors**
| **retourner** la feuille avec l'étiquette la plus représentée du parent.
fin
si $H(S) = 0$ **alors**
| **retourner** la feuille avec la prédiction.
fin
pour i allant de 0 à $|Q| - 1$ **faire**
| Calculer $\sum_{S_k} \frac{|S_k|}{|S|} H(S_k)$ pour la question q_i;
fin
Choisir i minimisant cette somme.
pour chaque S_k **faire**
| $f_k =$ID3$(S_k, Q \setminus \{q_i\})$
fin
retourner Noeud$(q_i, f_1, ..., f_k)$

Si on a un nombre de coordonnées très élevé, il est probable que l'une des questions soit très bonne.

Apprentissage non-supervisé.

Exemple 10: Partitionnement de données.

Étant donné un jeu de données, le partitionner en sous-ensembles.

Exemple 11: Réduction des dimensions.

Étant donné un jeu de données et un nombre de dimensions d, chercher à projeter le jeu de données dans un espace de plus faible dimension.

Définition 12

Pour l'apprentissage non-supervisé, il n'y a pas spécialement de phase de test puisqu'on ne réalise pas de prédiction.

Exemple 13: Algorithme des k moyennes.

Algorithme 2 : k moyennes.

Entrées : Un jeu de données S et un entier k.
$T \leftarrow$ liste de k éléments de S.
tant que T a été modifié **faire**
  pour chaque élément de S **faire**
    Calculer son voisin le plus proche dans T.
  fin
  pour chaque sous-ensemble ainsi créé **faire**
    Calculer le nouveau représentant du sous-ensemble : son barycentre.
  fin
fin

Il peut sembler que l'algorithme ne termine jamais, mais ce n'est pas le cas, puisqu'on admet la terminaison. (Un variant de boucle est la somme des carrés des distances aux représentants)
Le choix des k premiers éléments peut avoir une grande incidence sur le résultat.
On peut choisir le premier élément aléatoirement puis choisir les suivants afin qu'ils soient le plus éloigné des précédents.
On peut essayer d'étiqueter les sous-ensembles, mais ce n'est pas toujours faisable.
Si k est trop élevé, on peut avoir des sous-ensembles très proches. Si k est trop faible, on peut avoir des sous-ensembles très hétérogènes. On préfère avoir k trop élevé et unir les groupes proches.
On peut compresser des images en définissant une palette de k couleurs par cette méthode.

Exemple 14: Classification hiérarchique ascendante.

La classification hiérarchique ascendante consiste à rassembler des sous-ensembles proches successivement jusqu'à avoir un seul sous-ensemble restant. Cela crée un arbre de fusions successives pour lequel on peut choisir la valeur de k après la construction de l'arbre.
On peut définir une distance entre deux sous-ensembles pour déterminer si ils sont proches, en calculant la distance entre leurs barycentres, ou en calculant la moyenne des distances.
On obtient alors un arbre des fusions appelé **dendogramme**. On peut alors choisir k. En biologie, on est amené à construire des arbres phylogénétiques pour classer les espèces en genre, ordres, sous-ordres.

Définition 15: Qualité du jeu de données.

Les algorithmes qu'on a vu sont plus simples que ceux utilisés en pratique. L'avantage qu'ils présente réside dans la simplicité du résultat après apprentissage. La plupart des algorithmes d'apprentissage ne sont pas adaptés à un problème spécifique, c'est le jeu de données qui leur permet de se spécialiser. Un mauvais jeu de données peut être causé par : des erreurs dans le jeu, une taille trop petite, trop de dimensions, un manque de représentativité... C'est la grande difficulté dans les méthodes d'apprentissage. La plupart des problèmes de biais viennent des jeux de données.